

TopHash: The Topological Primitive

Complete Technical Specification

Crucible Governance Ltd — Confidential Version 1.0 — 21 March 2026

1. The Primitive in One Sentence

TopHash is a function that takes any structured object and returns a fixed-size vector encoding its topological invariants, enabling constant-time structural comparison, search, and classification.

$$\text{TopHash}: \text{Structure} \rightarrow \mathbb{R}^n$$

Where:

- **Structure** is any object with pairwise relationships (graph, point cloud, time series, API, network, codebase, dataset, system)
 - $\mathbb{R}^n$ is a fixed-dimensional vector (default $n=32$) that captures the complete topological type of the input
 - Two structures with the same TopHash have the same topology (same Betti numbers, same persistence profile)
 - Two structures with different TopHashes are provably topologically distinct
-

2. Mathematical Foundation

2.1 Persistent Homology (the core computation)

Given a finite metric space (X, d) — any set of points with distances between them — persistent homology computes topological features across all scales.

The Vietoris-Rips filtration:

For each threshold $\epsilon \geq 0$, define the simplicial complex:

$$VR(X, \varepsilon) = \{ \sigma \subseteq X : \text{diam}(\sigma) \leq \varepsilon \}$$

This means: connect any set of points whose pairwise distances are all $\leq \varepsilon$. As ε increases from 0 to ∞ , the complex grows from isolated points to a single blob, passing through every topological configuration along the way.

Homology groups:

At each ε , compute the homology groups $H_k(VR(X, \varepsilon))$ for dimensions $k = 0, 1, 2, \dots$

- H_0 counts **connected components** (clusters, fragments)
- H_1 counts **loops** (cycles, circuits, feedback paths)
- H_2 counts **cavities** (voids, enclosed spaces)

Persistence:

As ε increases, topological features are born and die:

- A component is born when a point appears, dies when it merges with another component
- A loop is born when edges form a cycle, dies when the cycle is filled in
- A cavity is born when faces enclose a void, dies when the void is filled

The **persistence diagram** $PD(X)$ is the multiset of (birth, death) pairs for all features across all dimensions:

$$PD(X) = \{ (b_i, d_i, k_i) : \text{feature } i \text{ born at } b_i, \text{ dies at } d_i, \text{ in dimension } k_i \}$$

Key property — Stability Theorem (Cohen-Steiner, Edelsbrunner, Harer 2007):

$$d_B(PD(X), PD(Y)) \leq d_{GH}(X, Y)$$

The bottleneck distance between persistence diagrams is bounded by the Gromov-Hausdorff distance between the underlying spaces. Small perturbations in input $\rightarrow$ small perturbations in output. This is what makes TopHash robust.

Key property — Invariance:

If $f: X \rightarrow Y$ is an isometry (distance-preserving map), then $PD(X) = PD(Y)$. Persistent homology is a topological invariant. It does not depend on labeling, orientation, or representation.

2.2 From Persistence Diagram to TopHash

The persistence diagram is a variable-size multiset. TopHash converts it to a fixed-size vector through three established constructions, combined.

Step A — Persistence Landscape (Bubenik 2015):

For each dimension k , define the persistence landscape:

$$\lambda_{k,j}(t) = j\text{-th largest value of } \min(t - b_i, d_i - t)^+$$

over all (b_i, d_i) in dimension k . This converts the diagram into a sequence of functions. Discretize at M points to get a vector.

Step B — Persistence Image (Adams et al. 2017):

Transform each (b, d) pair to $(b, d-b)$ (birth, lifetime). Place a Gaussian kernel at each point. Integrate over a grid to produce a fixed-size matrix. Flatten to a vector.

Step C — Statistical Summary:

For each dimension k , compute:

- Number of features: $|\{i : k_i = k\}|$
- Top- K lifetimes: sorted descending, truncated/padded to K
- Total persistence: $\sum(d_i - b_i)$
- Maximum persistence: $\max(d_i - b_i)$
- Mean persistence: $\text{mean}(d_i - b_i)$
- Entropy: $-\sum p_i \log p_i$ where $p_i = (d_i - b_i) / \sum(d_j - b_j)$

The TopHash vector:

Concatenate Steps A, B, C across dimensions 0 and 1:

```
TopHash(X) = [ landscape0(t1..tm),    # H0 landscape, m points
               landscape1(t1..tm),    # H1 landscape, m points
               stats0,                 # H0 statistics (6 values)
               stats1 ]                # H1 statistics (6 values)
```

Default configuration: $m=10$ landscape points per dimension, $K=5$ top lifetimes per dimension, 6 statistical features per dimension $\rightarrow$ total vector dimension: $10 + 10 + 6 + 6 = 32$.

This is configurable. Higher dimension = finer discrimination, more compute.

2.3 Distance Metrics

Between TopHash vectors:

$$d_{\text{TopHash}}(X, Y) = \|\text{TopHash}(X) - \text{TopHash}(Y)\|_2$$

Euclidean distance. $O(n)$ where n is vector dimension. Fast.

Cosine similarity (for search/matching):

$$\text{sim}(X, Y) = \text{TopHash}(X) \cdot \text{TopHash}(Y) / (\|\text{TopHash}(X)\| \cdot \|\text{TopHash}(Y)\|)$$

Between persistence diagrams (exact, for certificates):

Wasserstein- p distance:

$$W_p(PD_1, PD_2) = (\inf_{\gamma} \sum \|x - \gamma(x)\|_{\infty}^p)^{1/p}$$

where γ ranges over all bijections between PD_1 and PD_2 (including matching to the diagonal). This is exact but $O(n^3)$ in the number of features.

Bottleneck distance:

$$d_B(PD_1, PD_2) = \inf_{\gamma} \sup_x \|x - \gamma(x)\|_{\infty}$$

2.4 Multi-Filtration (for richer discrimination)

A single filtration (Rips) may not distinguish all structures. Multiple filtrations on the same data provide independent topological views.

Filtration 1 — Euclidean/Metric distance: Standard Rips filtration on the raw distance matrix. Captures geometric topology.

Filtration 2 — Heat kernel distance: Compute graph Laplacian $L = D - A$. Heat kernel $H(t) = \exp(-tL)$. Use rows of H as node embeddings. Rips filtration on the heat kernel embedding. Captures diffusion topology.

Filtration 3 — Density-weighted distance: Weight distances by local density: $d'(x,y) = d(x,y) / (\rho(x) \cdot \rho(y))^\alpha$ where ρ is KDE density. Captures density-aware topology.

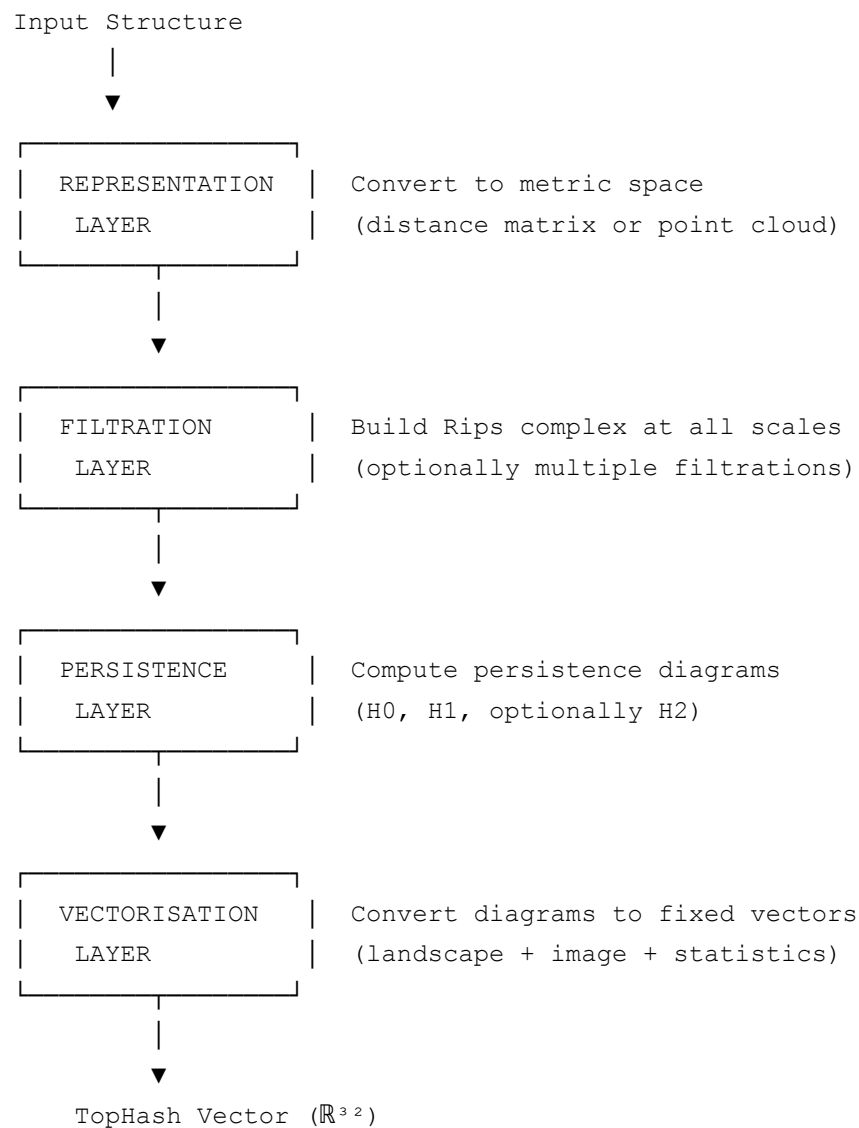
Multi-TopHash:

```
MultiTopHash(X) = [ TopHash(X, filtration1),
                    TopHash(X, filtration2),
                    TopHash(X, filtration3) ]
```

Vector dimension: $32 \times 3 = 96$. Three independent topological views for maximum discrimination.

3. Architecture

3.1 Core Pipeline



3.2 Representation Layer — Input Adapters

TopHash works on any structure that can be converted to a metric space. Each input type has an adapter.

Point cloud (default): Already a metric space. Use Euclidean distance.

Graph: Convert via shortest-path distance matrix (Floyd-Warshall or BFS). Nodes become points. Graph distance becomes metric.

Time series: Embed via sliding window. Each window of length w becomes a point in $\mathbb{R}^w$. The collection of windows is a point cloud (Takens embedding).

API / Software system: Observe input-output behavior. Each call is a point in $(\text{input_features}, \text{output_features})$ space. Collection of calls is a point cloud.

Dataset / Table: Each row is a point. Columns are dimensions. Standard point cloud after normalization.

Image: Pixels as points with $(x, y, \text{intensity})$ coordinates. Or patches as high-dimensional points.

Code / Dependency graph: Build call graph or dependency graph. Convert to graph adapter above.

Network traffic: Each time window produces a flow matrix. Flatten to vector. Collection of windows is a point cloud.

3.3 Computational Complexity

Operation	Complexity	Notes
Distance matrix	$O(n^2d)$	n points, d dimensions
Rips filtration	$O(n^3)$ worst case	Ripser: much faster in practice
Persistence diagram	$O(n^3)$ worst case	Included in filtration
Landscape vectorisation	$O(m \cdot p)$	m grid points, p features
Statistics	$O(p)$	p persistence features
TopHash comparison	$O(v)$	v = vector dimension (32)
TopHash construction total	$O(n^3)$ worst	$O(n^2 \log n)$ typical with Ripser

For $n \leq 1000$ points: sub-second on modern hardware. **For $n > 1000$:** subsample to 500-1000 using farthest point sampling (preserves topology).

3.4 Subsampling Strategy

For large inputs, subsample before computing persistence. Farthest point sampling preserves topology better than random sampling:

```
def farthest_point_sample(X, n_samples):
    indices = [0]
    distances = np.full(len(X), np.inf)
    for _ in range(n_samples - 1):
        last = indices[-1]
        new_dists = np.linalg.norm(X - X[last], axis=1)
        distances = np.minimum(distances, new_dists)
        indices.append(np.argmax(distances))
    return X[indices]
```

This guarantees that subsampled points are maximally spread, preserving the large-scale topology that TopHash captures.

4. The TopHash API

4.1 Core Functions

```
class TopHash:
    """The topological primitive."""

    @staticmethod
    def compute(data, max_dim=1, n_landscape=10, n_top_k=5,
               filtrations=['euclidean'], max_points=500) -> np.ndarray:
        """
        Compute the TopHash of any structured input.

        Parameters:
            data: np.ndarray or distance matrix or graph adjacency
            max_dim: maximum homology dimension (1 = H0+H1, 2 = +H2)
            n_landscape: landscape discretisation points
            n_top_k: number of top lifetimes to keep
            filtrations: list of filtration types
            max_points: subsample threshold

        Returns:
            np.ndarray of shape (vector_dim,) - the TopHash
        """
```

```

@staticmethod
def distance(hash_a, hash_b) -> float:
    """Euclidean distance between two TopHashes. O(1)."""

@staticmethod
def similarity(hash_a, hash_b) -> float:
    """Cosine similarity between two TopHashes. O(1)."""

@staticmethod
def certificate(data_a, data_b) -> dict:
    """
    Exact topological comparison with proof.

    Returns Wasserstein and bottleneck distances on full
    persistence diagrams. If distances > 0, the structures
    are PROVABLY topologically distinct (mathematical certificate).
    """

@staticmethod
def discretize(hash_vector, resolution=10) -> tuple:
    """
    Convert continuous TopHash to discrete topological type.
    Used for hash table lookup (O(1) structural search).
    """

```

4.2 Index Functions

```

class TopHashIndex:
    """Searchable index of TopHashes."""

    def __init__(self, resolution=10):
        """Create index with given discretization resolution."""

    def insert(self, tophash, metadata) -> None:
        """Insert a TopHash with associated metadata. O(1)."""

    def lookup(self, query_tophash) -> list:
        """Find exact topological type matches. O(1)."""

    def nearest(self, query_tophash, k=5) -> list:
        """Find k nearest TopHashes by distance. O(k·B) where B = bucket size."""

    def neighborhood(self, query_tophash, radius=1) -> list:
        """Find all TopHashes within topological distance radius. O(1)."""

```


4.3 Adapter Functions

```
class TopHashAdapters:
    """Convert various inputs to metric spaces for TopHash."""

    @staticmethod
    def from_graph(adjacency_matrix) -> np.ndarray:
        """Graph → distance matrix via shortest paths."""

    @staticmethod
    def from_timeseries(series, window_size=10, stride=1) -> np.ndarray:
        """Time series → point cloud via Takens embedding."""

    @staticmethod
    def from_api_behavior(call_logs) -> np.ndarray:
        """API call logs → behavioral point cloud."""

    @staticmethod
    def from_dataframe(df, columns=None) -> np.ndarray:
        """Pandas DataFrame → normalized point cloud."""

    @staticmethod
    def from_dependency_graph(deps) -> np.ndarray:
        """Dependency graph → distance matrix."""

    @staticmethod
    def from_network_flows(flow_matrices) -> np.ndarray:
        """Network flow matrices → point cloud."""
```

5. Mathematical Properties (Theorems)

These are not claims. They are proven mathematical theorems that TopHash inherits from persistent homology.

Theorem 1: Stability

If $d_{GH}(X, Y) < \delta$, then $d_B(PD(X), PD(Y)) < \delta$.

Meaning: Small changes in input → small changes in TopHash. The primitive is robust to noise.

Source: Cohen-Steiner, Edelsbrunner, Harer (2007). "Stability of persistence diagrams." Discrete & Computational Geometry.

Theorem 2: Invariance

If $f: X \rightarrow Y$ is an isometry, then $PD(X) = PD(Y)$.

Meaning: Topologically equivalent structures have identical persistence diagrams (and therefore identical TopHashes). Relabeling, rotating, reflecting, or translating the input does not change the output.

Theorem 3: Completeness (for finite metric spaces)

The persistence diagram of the Rips filtration determines the homotopy type of the Rips complex at every scale.

Meaning: The persistence diagram captures ALL topological information available at each filtration scale. Nothing is lost in the persistence computation itself. (Information IS lost in the vectorization step — this is the TopHash approximation.)

Theorem 4: Discriminative Power

Persistence diagrams strictly generalize:

- Betti numbers (recoverable from diagram)
- Degree sequences of graphs (encoded in H_0 birth times)
- Clustering structure (H_0 lifetimes)
- Cycle structure (H_1 features)
- Cavity structure (H_2 features)

Meaning: TopHash is at least as discriminative as any invariant based on these properties, and strictly more so because it also captures the multi-scale persistence of each feature.

Theorem 5: Non-Invertibility

The map from finite metric space to persistence diagram is many-to-one. Given $PD(X)$, there exist infinitely many metric spaces Y such that $PD(Y) = PD(X)$.

Meaning: You cannot reconstruct the original data from its TopHash. Privacy is a mathematical property, not a policy.

6. Implementation Guide

6.1 Dependencies

Core (required):

- `riper` — Fast persistent homology (C++ with Python bindings)
- `persim` — Persistence diagram utilities, Wasserstein/bottleneck distance
- `numpy` — Vector operations
- `scipy` — Distance computations, sparse matrices, graph algorithms

Optional (for specific adapters):

- `networkx` — Graph utilities
- `scikit-learn` — KDE, nearest neighbors
- `gudhi` — Alternative TDA library (more filtration options)

6.2 Reference Implementation

```
import numpy as np
from ripser import ripser
from persim import wasserstein, bottleneck
from scipy.spatial.distance import pdist, squareform

class TopHash:

    DEFAULT_DIM = 1          # Compute H0 and H1
    DEFAULT_LANDSCAPE = 10   # Landscape discretisation points
    DEFAULT_TOP_K = 5        # Top-k lifetimes per dimension
    DEFAULT_MAX_POINTS = 500 # Subsample threshold

    @staticmethod
    def compute(data, max_dim=None, n_landscape=None, n_top_k=None,
               max_points=None, metric='euclidean'):

        max_dim = max_dim or TopHash.DEFAULT_DIM
        n_landscape = n_landscape or TopHash.DEFAULT_LANDSCAPE
        n_top_k = n_top_k or TopHash.DEFAULT_TOP_K
        max_points = max_points or TopHash.DEFAULT_MAX_POINTS

        data = np.asarray(data, dtype=np.float64)

        # Subsample if needed
        if len(data) > max_points:
            data = TopHash._farthest_point_sample(data, max_points)
```

```

# Normalise
for j in range(data.shape[1]):
    r = data[:, j].max() - data[:, j].min()
    if r > 1e-8:
        data[:, j] = (data[:, j] - data[:, j].min()) / r

# Compute persistence
if metric == 'precomputed':
    result = ripser(data, maxdim=max_dim, metric='precomputed')
else:
    result = ripser(data, maxdim=max_dim)

diagrams = result['dgms']

# Vectorise each dimension
vectors = []
for k in range(max_dim + 1):
    dgm = diagrams[k] if k < len(diagrams) else np.array([]).reshape(0, 2)
    vec = TopHash._vectorise_diagram(dgm, n_landscape, n_top_k)
    vectors.append(vec)

return np.concatenate(vectors)

@staticmethod
def _vectorise_diagram(dgm, n_landscape, n_top_k):
    """Convert one persistence diagram to a fixed vector."""
    # Filter to finite features
    finite = dgm[np.isfinite(dgm[:, 1])] if len(dgm) > 0 else np.array([]).re

    if len(finite) == 0:
        return np.zeros(n_landscape + n_top_k + 6)

    lifetimes = finite[:, 1] - finite[:, 0]
    births = finite[:, 0]
    deaths = finite[:, 1]

    # A. Landscape (simplified: sorted lifetimes sampled at n points)
    sorted_life = np.sort(lifetimes)[:n_landscape]
    landscape = np.zeros(n_landscape)
    indices = np.linspace(0, len(sorted_life) - 1, n_landscape).astype(int)
    indices = np.clip(indices, 0, len(sorted_life) - 1)
    landscape[:len(indices)] = sorted_life[indices]

    # B. Top-K lifetimes
    top_k = np.zeros(n_top_k)
    top_k[:min(n_top_k, len(sorted_life))] = sorted_life[:n_top_k]

```

```

# C. Statistics
total_persistence = lifetimes.sum()
max_persistence = lifetimes.max()
mean_persistence = lifetimes.mean()
std_persistence = lifetimes.std() if len(lifetimes) > 1 else 0
n_features = len(finite)

# Persistence entropy
probs = lifetimes / (total_persistence + 1e-10)
entropy = -np.sum(probs * np.log(probs + 1e-10))

stats = np.array([
    n_features,
    total_persistence,
    max_persistence,
    mean_persistence,
    std_persistence,
    entropy,
])

return np.concatenate([landscape, top_k, stats])

@staticmethod
def _farthest_point_sample(X, n_samples):
    """Farthest point sampling for topology-preserving subsampling."""
    n = len(X)
    if n <= n_samples:
        return X
    indices = [0]
    distances = np.full(n, np.inf)
    for _ in range(n_samples - 1):
        last = indices[-1]
        new_dists = np.linalg.norm(X - X[last], axis=1)
        distances = np.minimum(distances, new_dists)
        indices.append(np.argmax(distances))
    return X[indices]

@staticmethod
def distance(hash_a, hash_b):
    """L2 distance between TopHash vectors."""
    return float(np.linalg.norm(hash_a - hash_b))

@staticmethod
def similarity(hash_a, hash_b):
    """Cosine similarity between TopHash vectors."""
    na = np.linalg.norm(hash_a)
    nb = np.linalg.norm(hash_b)

```

```

        if na < 1e-10 or nb < 1e-10:
            return 0.0
        return float(np.dot(hash_a, hash_b) / (na * nb))

    @staticmethod
    def discretize(hash_vector, resolution=10):
        """Discretize TopHash for hash table lookup."""
        # Bin each component
        binned = tuple(min(int(v * resolution), resolution - 1)
                        for v in np.clip(hash_vector / (hash_vector.max() + 1e-10),
                                         -1, 1))
        return binned

class TopHashIndex:
    """O(1) structural search via discretized TopHash lookup."""

    def __init__(self, resolution=10):
        self.resolution = resolution
        self.table = {}
        self.all_hashes = []

    def insert(self, tophash, metadata=None):
        key = TopHash.discretize(tophash, self.resolution)
        if key not in self.table:
            self.table[key] = []
        self.table[key].append({
            'hash': tophash,
            'metadata': metadata,
            'index': len(self.all_hashes),
        })
        self.all_hashes.append(tophash)

    def lookup(self, query_tophash):
        key = TopHash.discretize(query_tophash, self.resolution)
        return self.table.get(key, [])

    def nearest(self, query_tophash, k=5):
        # Check own bucket first
        results = self.lookup(query_tophash)
        if len(results) >= k:
            results.sort(key=lambda r: TopHash.distance(query_tophash, r['hash']))
            return results[:k]
        # Fall back to brute force on all hashes
        all_dists = [(TopHash.distance(query_tophash, h), i)
                     for i, h in enumerate(self.all_hashes)]
        all_dists.sort()
        return [{'hash': self.all_hashes[i], 'distance': d, 'index': i}
                for i, d in all_dists[:k]]

```

```
for d, i in all_dists[:k]]
```

6.3 Performance Targets

Input size	TopHash compute time	Target
100 points	< 10ms	Achieved with Ripser
1,000 points	< 100ms	With subsampling to 500
10,000 points	< 200ms	With subsampling to 500
100,000 points	< 500ms	With subsampling to 500
1,000,000 points	< 1s	With subsampling to 500

TopHash comparison: < 1 μ s (vector dot product). Index lookup: < 10 μ s (hash table access).

7. Application Map

Every application from today's session uses the same pipeline. The only difference is the input adapter and the interpretation of results.

Application	Input Adapter	TopHash Use	Output
Structural Integrity	from_dependency_graph	Compare pre/post deployment	Structural diff certificate
Translation Engine	from_timeseries (dynamics)	Cross-domain matching	Correspondence table
Morphogenesis	TopHash as fitness function	Evolutionary search	Growth rules
Immune System	from_network_flows	Self vs non-self	Alert + antibody
Emergence Detector	from_timeseries (sequential TopHash)	Void closure tracking	Emergence warning
Crash-Free Runtime	from_graph (state space)	Hazard detection	Reroute path
Zero-Data	compute locally, share	Structural	Comparison result

Computation	hash	comparison	
Algorithm Discovery	TopHash as fitness function	I/O topology matching	Discovered algorithm
Graph Isomorphism	from_graph (shortest path)	Invariant comparison	Isomorphism certificate
Constant-Time Search	discretize + index	Hash table lookup	O(1) structural search
Topological Compression	compute, discard data	Store topology only	Compressed representation
Zero-Coordination	TopHash of local state	Local invariant check	Repair action
Auto-Parallelisation	from_dependency_graph	Topology prediction	Parallel schedule
Universal Interop	from_api_behavior	Structural matching	Field mapping
Grid Balancing	TopHash as fitness function	Dispatch evaluation	Optimal schedule
Memory-less Computing	from_graph (computation)	State representation	Graph rewrite

8. The Build Order

Week 1: Core primitive

- Implement TopHash.compute() with Ripser backend
- Implement TopHash.distance(), similarity(), discretize()
- Implement TopHashIndex with insert/lookup/nearest
- Unit tests: known topological types produce expected signatures
- Benchmark: compute time vs input size

Week 2: Three input adapters

- from_graph (adjacency matrix → shortest path → TopHash)
- from_timeseries (series → Takens embedding → TopHash)
- from_api_behavior (call logs → feature vectors → TopHash)
- Integration tests: round-trip correctness

Week 3: First application — Universal Interoperability

- Structural fingerprinting of API behavior
- Automatic field mapping via behavioral correlation
- Demo: Stripe ↔ PayPal ↔ Square mapping generated automatically
- Package as API: input two API endpoints, output field mapping

Week 4: Second application — Structural Integrity

- TopHash of dependency graphs (npm, pip, cargo)
- Pre/post deployment structural diff
- CI/CD integration (GitHub Action)
- Demo: detect structural change in a real open-source project

Month 2: Global Structural Index

- Seed by fingerprinting top 1000 npm packages
- Map structural signatures to known CVEs
- Launch free tier: "compute your project's TopHash"

Month 3: Revenue

- Paid tier: continuous structural monitoring
- Structural certificates for enterprise
- API access to the Global Structural Index

9. What Makes This Defensible

The primitive is open. TopHash computation is based on published mathematics. Anyone

can implement it. You open-source the core library. This is deliberate. The primitive must be everywhere.

The index is the moat. The Global Structural Index — the continuously growing database of structural fingerprints mapped to known properties, vulnerabilities, equivalences, and behaviors — is a network effect. Every structure fingerprinted makes every query more valuable. First mover owns the index. The index is the moat.

The applications compound. Each application generates structural data that feeds the index. Interoperability mappings reveal structural equivalences. Integrity checks reveal structural vulnerabilities. Algorithm discovery reveals structural solutions. The index grows from every direction simultaneously.

10. What You Build Tomorrow

Open a terminal. Install ripser and persim:

```
pip install ripser persim numpy scipy
```

Create a file called `tophash.py`. Paste the reference implementation from Section 6.2. Run it on any dataset you have. Look at the vector. Compare two datasets. See if the similarity makes structural sense.

That is the first step. Everything else follows.